

Experiment-3-Intermediate-1

Title: FPGA-Based Adaptive-Precision MAC Accelerator for Energy-Efficient Computing

Objective:

The objective of this project is to design and verify a Multiply–Accumulate (MAC) unit in Verilog HDL, which is the fundamental arithmetic operation of digital signal processing and neural-network hardware. The design multiplies two signed operands and adds the product to a running accumulator, using a two-stage pipeline of a multiplier register and an accumulator register.

The design supports selectable operand precision (4-bit, 8-bit, 16-bit and 32-bit) through a precision controller that sign-extends the operands to the 32-bit datapath width. A top-level module provides a board-style interface in which two operands and the precision are entered with four switches and one push button, and the two least significant bits of the result are shown on two LEDs. The design is validated through ModelSim RTL simulation using one testbench for the MAC core and one for the complete top-level module.

Block Diagram:

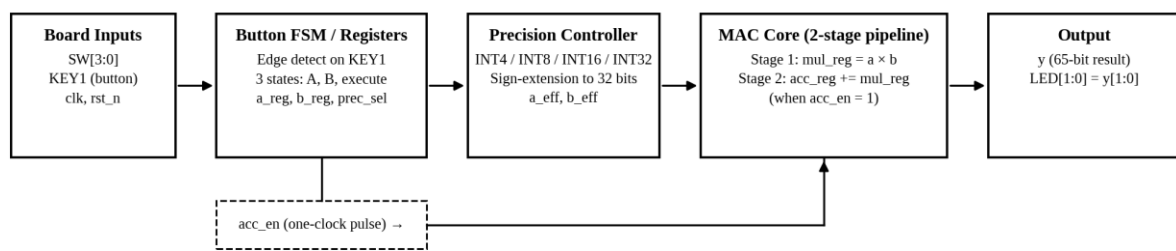


Fig. 1. Overall architecture of the multi-precision MAC unit.

The operands are entered through the four switches SW[3:0] and the push button KEY1. A small three-state controller in the top-level module detects each press of KEY1: the first press captures operand A, the second captures operand B, and the third selects the precision from SW[1:0] and issues a one-clock accumulate-enable pulse (acc_en). The captured operands pass through the precision controller, which selects the lower 4, 8, 16 or 32 bits of each operand and sign-extends it to 32 bits. The MAC core multiplies the two effective operands in its first pipeline stage and, when acc_en is high, adds the registered product to the accumulator in its second stage. The accumulator value is registered at the output y, and its two least significant bits drive LED[1:0].

RTL Design:

The MAC is described in Verilog HDL using the mac_top.v, mac_core.v and precision_ctrl.v modules. Two testbenches are used for functional verification: mac_core_tb.v for the core datapath and mac_top_tb_final.v for the complete top-level design.

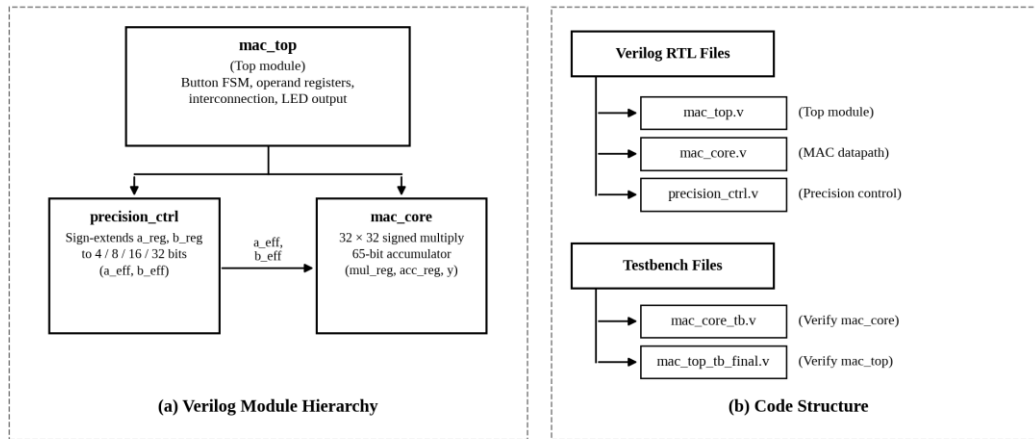


Fig. 2. Verilog module hierarchy and code structure of the MAC unit.

1. mac_core.v

`mac_core` implements the multiply-accumulate datapath with a parameterised data width ($DATA_W = 32$). It takes two signed $DATA_W$ -bit inputs a and b and an accumulate-enable input acc_en , and produces a signed $(2 \cdot DATA_W + 1)$ -bit output y (65 bits by default), which is wide enough to hold the 64-bit product plus one carry bit. The design is a two-stage pipeline. In the first stage the product $a \times b$ is registered in `mul_reg` every clock cycle. In the second stage, when acc_en is high, `mul_reg` is added to the accumulator `acc_reg`. The output y is a registered copy of `acc_reg`, and all registers are cleared by the active-low asynchronous reset. Because of the pipeline registers, the result appears at y a few clock cycles after the operands are applied.

2. precision_ctrl.v

`precision_ctrl` selects the operand width using the 2-bit input `prec_sel`. For $2'b00$ (INT4), $2'b01$ (INT8) and $2'b10$ (INT16), the lower 4, 8 or 16 bits of each input are taken and sign-extended to the full 32-bit width by replicating the most significant bit of the selected field; for $2'b11$ (INT32) the inputs pass through unchanged. The outputs `a_eff` and `b_eff` therefore always carry a correctly signed value of the selected precision, so the same 32-bit multiplier is used for all four precisions.

3. mac_top.v

`mac_top` is the board-level top module. It instantiates the precision controller and the MAC core and implements a three-state demonstration controller driven by rising-edge detection of `KEY1`. In state 0 a press stores the four switch values into operand register `a_reg`; in state 1 a press stores them into `b_reg`; and in state 2 a press loads `prec_sel` from `SW[1:0]`, issues a one-clock `acc_en` pulse, and returns to state 0. The `LED[1:0]` output is updated every cycle from the two least significant bits of the MAC result, and the registers are cleared by the active-low reset. Since the operands come from four switches, the values are limited to 0–15 on the board.

4. Testbench and Verification

`mac_core_tb` drives the MAC core directly with a 10 ns clock. After releasing reset it applies the operand pairs 3×4 , 2×5 and 1×6 with `acc_en` toggled, so that the accumulation and the hold behaviour of the accumulator can be observed on y . This testbench has no automatic checks; the output is verified from the waveform.

mac_top_tb_final verifies the complete top-level design through its board-style ports. After reset, it sets SW = 3 and presses KEY1 to capture A = 3, repeats the step to capture B = 3, and then sets SW = 0000 (INT4) and presses KEY1 a third time to execute. After waiting five clock cycles for the pipeline, it prints the precision, the effective operands, mul_reg, acc_reg, mac_result and the LED value, and checks automatically that a_reg = 3, b_reg = 3 and acc_reg = 9. A PASS or FAIL message is printed accordingly.

Simulation Results:

The RTL was simulated in ModelSim (Intel FPGA Starter Edition 10.5b). Compilation of all files completed with 0 errors (Fig. 4). Fig. 3 shows the result of mac_core_tb. After reset is released, the products $3 \times 4 = 12$ and $2 \times 5 = 10$ are accumulated, and the output y reaches 12 and then 22. When 1×6 is applied with acc_en held high, the product 6 is added on successive clock cycles, and y rises to 28 and then 34. This confirms the multiply, accumulate and output-register behaviour of the core, including the pipeline delay of the result with respect to the inputs.

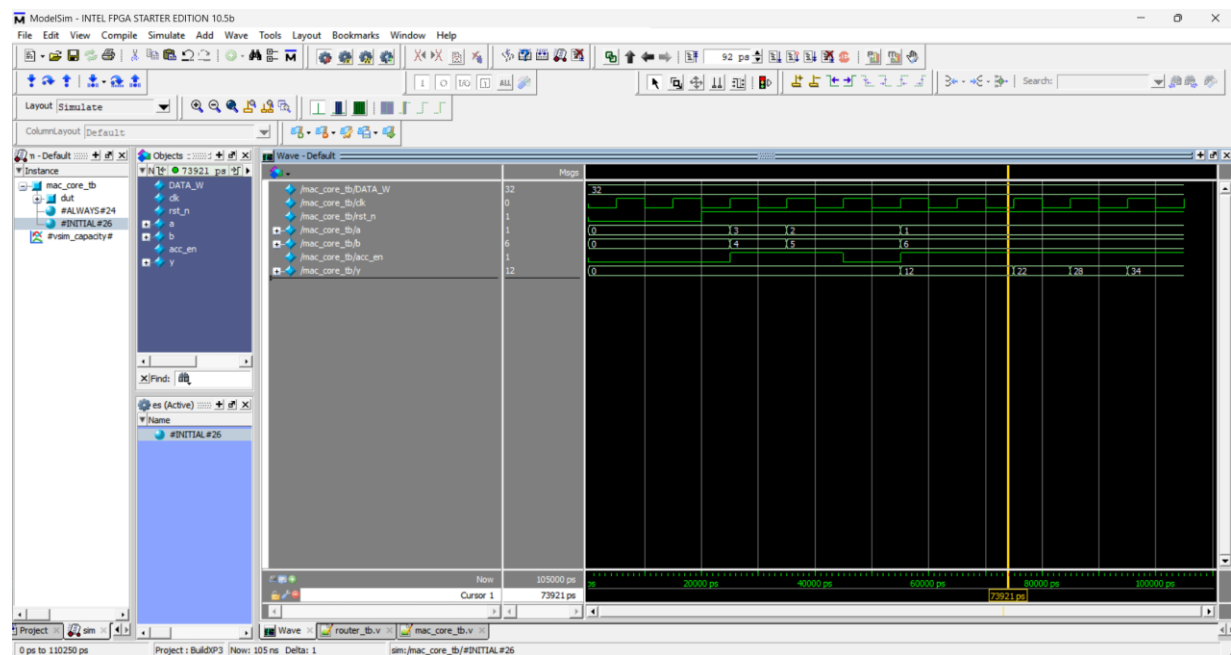


Fig. 3. ModelSim waveform of the MAC core testbench (mac_core_tb).

```

# Compile of mac_core_tb.v was successful.
# Compile of mac_top_tb.v was successful.
# Compile of mac_top.v was successful.
# Compile of mac_core.v was successful.
# Compile of precision_ctrl.v was successful.
# 5 compiles, 0 failed with no errors.
ModelSim> vsim -gui work.mac_core_tb
# vsim -gui work.mac_core_tb
# Start time: 23:41:50 on Oct 10, 2026
# Loading work.mac_core_tb
# Loading work.mac_core
add wave -position insertpoint \
sim:/mac_core_tb/DATA_W \
sim:/mac_core_tb/clk \
sim:/mac_core_tb/rst_n \
sim:/mac_core_tb/a \
sim:/mac_core_tb/b \
sim:/mac_core_tb/acc_en \
sim:/mac_core_tb/y
VSIModel> run -all
** Note: tfinish : C:/Users/pgkri/Downloads/mac_core_tb.v(54)
# Time: 105 ns Iteration: 1 Instance: /mac_core_tb
# 1
# Break in Module mac_core_tb at C:/Users/pgkri/Downloads/mac_core_tb.v line 44
VSIModel>

```

Fig. 4. ModelSim transcript showing successful compilation of the design and testbench files.

Fig. 5 shows the waveform of the top-level testbench. SW is set to 0011 for the first two KEY1 pulses, which capture A = 3 and B = 3, and to 0000 for the third pulse, which selects INT4 precision and executes the MAC operation. After the pipeline delay, LED changes from 00 to 01, which corresponds to the two least significant bits of the result 9 (binary 1001).

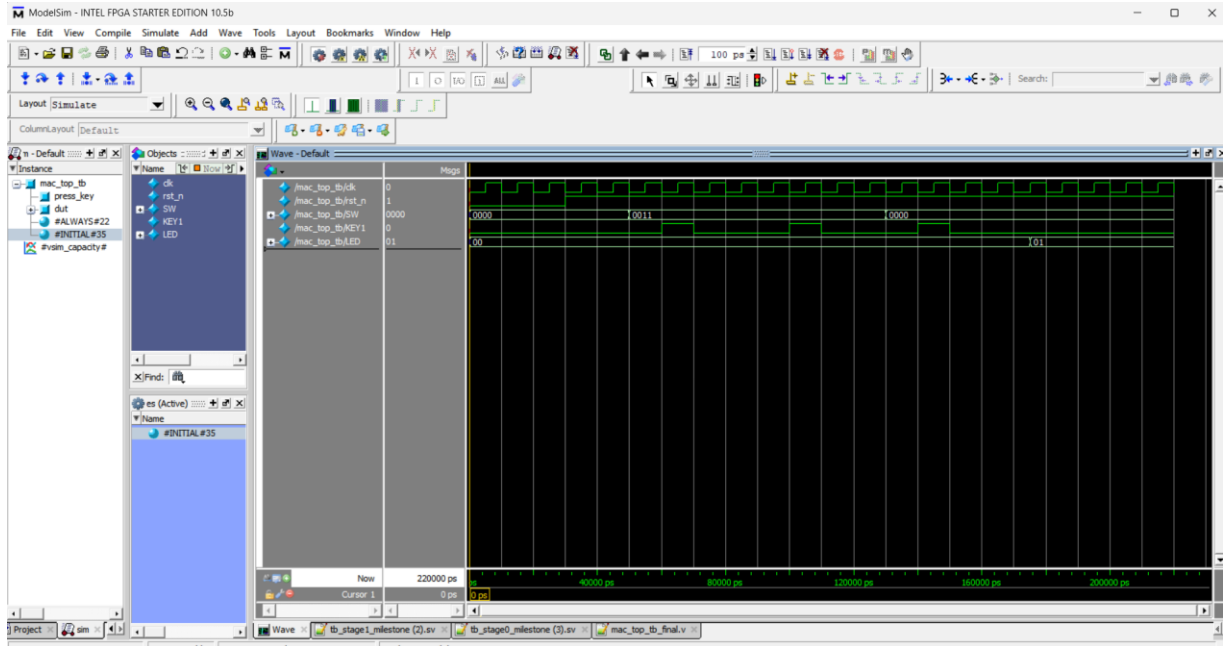


Fig. 5. ModelSim waveform of the top-level testbench (mac_top_tb).

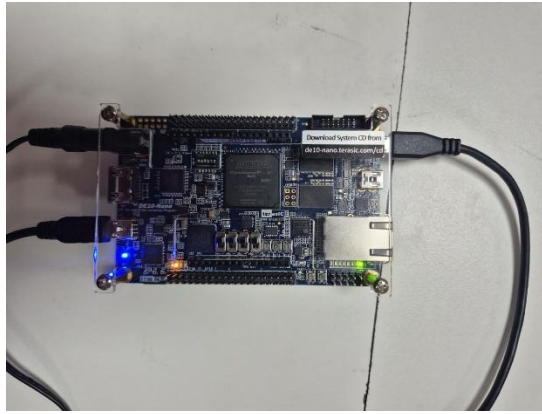
The transcript in Fig. 6 shows the values printed by the testbench: A is captured as 3 at 90 ns and B as 3 at 130 ns. At 220 ns the precision is 00 (INT4) with a_eff = 3 and b_eff = 3, and mul_reg = 9, acc_reg = 9, mac_result = 9 and LED = 01. The testbench prints PASS: 3 x 3 accumulated to 9, and the simulation ends at 220 ns.

```
ModelSim> vsim -gui work.mac_top_tb
# End time: 00:00:37 on Oct 11, 2026, Elapsed time: 0:03:55
# Errors: 0, Warnings: 0
# vsim -gui work.mac_top_tb
# Start time: 00:00:37 on Oct 11, 2026
# Loading work.mac_top_tb
# Loading work.mac_top
# Loading work.precision_ctrl
# Loading work.mac_core
add wave -position insertpoint \
sim:/mac_top_tb/clk \
sim:/mac_top_tb/rst_n \
sim:/mac_top_tb/SW \
sim:/mac_top_tb/KEY1 \
sim:/mac_top_tb/LED
VSI10> run -all
# [90 ns] A captured = 3
# [130 ns] B captured = 3
# [220 ns] precision=00 a_eff=3 b_eff=3
# [220 ns] mul_reg=9 acc_reg=9 mac_result=9 LED=01
# PASS: 3 x 3 accumulated to 9.
# ** Note: $finish : C:/Users/pgkri/Downloads/mac_top_tb_final.v(78)
# Time: 220 ns Iteration: 1 Instance: /mac_top_tb
# 1
# Break in Module mac_top_tb at C:/Users/pgkri/Downloads/mac_top_tb_final.v line 78
```

Fig. 6. ModelSim transcript of the top-level testbench showing the PASS result.

Hardware Implementation:

The top-level module mac_top.v is written with the ports of the target FPGA board (clk, rst_n, SW[3:0], KEY1 and LED[1:0]); the pin assignments are kept in the Quartus project. On the board, operand A and operand B are entered by setting the switches and pressing KEY1 once for each; the third press selects the precision from SW[1:0] and executes the operation. The two LEDs show the two least significant bits of the accumulated result, so for $3 \times 3 = 9$ the LEDs show 01. The results reported in this experiment are from RTL simulation; FPGA compilation and board testing are separate steps.



Applications:

The MAC operation is the core computation of digital filters, convolution, matrix multiplication, and the fully connected and convolutional layers of neural networks. A MAC with selectable precision allows the same hardware to be used at low precision (INT4 or INT8) for energy-efficient neural-network inference and at higher precision (INT16 or INT32) for signal-processing tasks that need a larger dynamic range.

The design can be used as a building block of larger arrays of processing elements, such as systolic arrays and neural-network accelerators on FPGAs, and as a basis for studying pipelining, accumulator sizing and mixed-precision arithmetic in embedded and edge-AI systems.

Conclusion:

The project successfully demonstrated a two-stage pipelined signed MAC unit with selectable INT4, INT8, INT16 and INT32 operand precision and a switch-and-button board interface. The RTL design was verified in ModelSim: the core testbench showed correct accumulation of 12, 22, 28 and 34, and the top-level testbench captured $A = 3$ and $B = 3$, executed in INT4 mode and accumulated 3×3 to 9, with the testbench reporting PASS. The work provided practical experience in Verilog RTL design, pipelined arithmetic, precision control and simulation-based verification.